🏆 SIH 2026 — Problem Statement 26027: WINNING BLUEPRINT

AI-Powered Automatic Block Planning to Maximize Asset Availability for Train Operations on Indian Railways

> **Team Motto:** *"Don't just solve the problem — redefine how India's railways think about maintenance."*

Table of Contents

1. 🎯 Winning Strategy
2. 📊 Winning PPT Design
3. 🛠️ Prototype Architecture — Out-of-Box Thinking
4. 📋 Step-by-Step Roadmap to Win the Finale
5. 🎤 Pitch Scripts: 2min / 5min / 10min
6. 🧪 Out-of-the-Box Test Cases & Solutions

🎯 1. WINNING STRATEGY

1.1 Think Like a Judge — What Wins and What Doesn't

Judging Criteria Breakdown (SIH 2026)

Criteria	Weight	What Judges ACTUALLY Look For
Problem Understanding	20%	Can you articulate the PAIN of a Section Controller at 2 AM trying to coordinate 3 departments? Do you know what a "corridor block" really is?
Innovation & Novelty	20–25%	NOT just "we used AI." They want to see a UNIQUE angle — Digital Twin? Reinforcement Learning? Game Theory for multi-department negotiation?
Technical Feasibility	20–25%	Clean architecture. Real algorithms. Not a ChatGPT wrapper. Show constraint satisfaction, optimization math, and scalable design.
Demo & Presentation	15–20%	A LIVE demo that works > 100 slides. Show a real block plan being generated in real-time on a map.
Completeness	10–15%	End-to-end flow: Data ingestion → AI Processing → Optimized Schedule → Dashboard → Alerts

The 5 Pillars of Our Winning Edge

```
┌─────────────────────────────────────────────────────────────────┐
│                    THE 5 WINNING PILLARS                        │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│  1. DOMAIN MASTERY          – Speak their language (IR terminology) │
│  2. AI DEPTH                – Real optimization, not buzzwords  │
│  3. SYSTEM INTEGRATION      – Mirror actual CRIS architecture   │
│  4. VISUAL IMPACT           – Digital Twin + Live Map Dashboard │
│  5. DEPLOYABILITY           – Show it can work TOMORROW         │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

1.2 Our Solution Name: **"RailMind AI"** 🧠🚂

> *Tagline: "From Disconnected Departments to Connected Intelligence"*

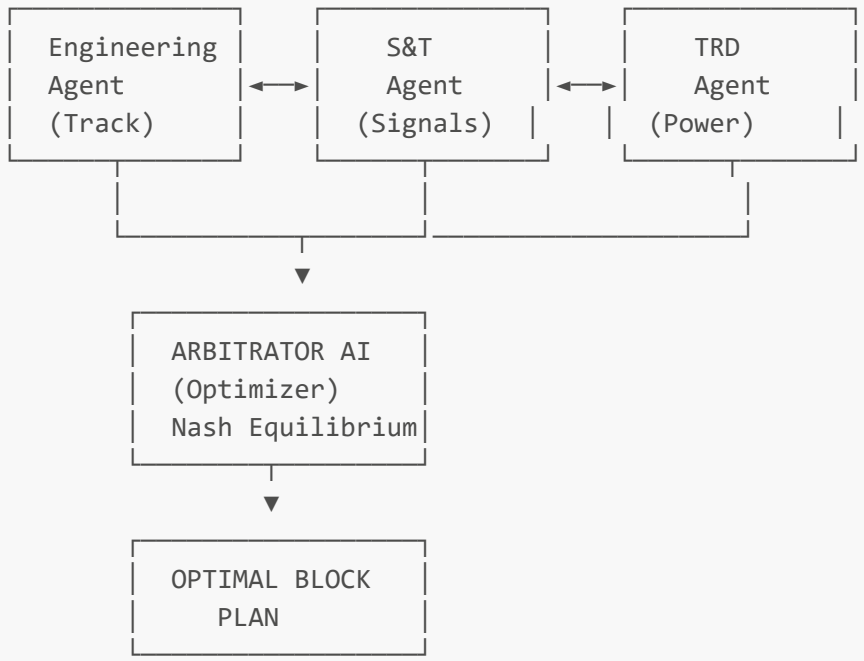
Why This Name Wins:

- **"Rail"** = Domain clarity
- **"Mind"** = AI/Intelligence
- Short, memorable, brandable — judges remember names

1.3 The Out-of-Box Differentiators (What Others WON'T Do)

🔥 Differentiator 1: Multi-Agent AI Negotiation System

Instead of a simple scheduler, we build **AI agents representing each department** (Engineering, S&T, TRD) that **negotiate** block windows using game-theoretic principles. This mirrors the real-world multi-stakeholder conflict.



Why judges will love this: It's not just scheduling — it's modeling real organizational behavior with AI. This shows deep problem understanding AND technical innovation.

🔥 Differentiator 2: Railway Digital Twin with Live Simulation

Build a **visual digital twin** of a railway section (e.g., Mumbai Division CSMT–Kalyan corridor) where judges can SEE:

- Trains moving in real-time
- Maintenance blocks being allocated/released
- Color-coded asset health degradation
- "What-if" scenario simulation

🔥 Differentiator 3: Predictive Maintenance Criticality Engine

Not just scheduling EXISTING maintenance — **predicting FUTURE failures** using:

- Track geometry degradation curves (TMS data patterns)
- Signal failure probability models (SMMS patterns)
- OHE wear patterns (TDMS patterns)

This transforms reactive → predictive → **prescriptive** maintenance.

🔥 Differentiator 4: NLP-Based Voice Command for Controllers

Railway controllers are busy people. Our system supports:

- "*RailMind, show me available blocks for track tamping on Kalyan-Kasara section next week*"
- "*Schedule a mega block for OHE maintenance on Sunday between KM 45-52*"

🔥 Differentiator 5: Conflict Resolution Explainability Dashboard

When the AI rejects or modifies a block request, it shows **WHY** with a visual explanation:

- "Block denied: Rajdhani Express passes KM 34 at 14:32. Suggested alternative: 10:00-12:30"
- Visual Gantt chart showing the conflict

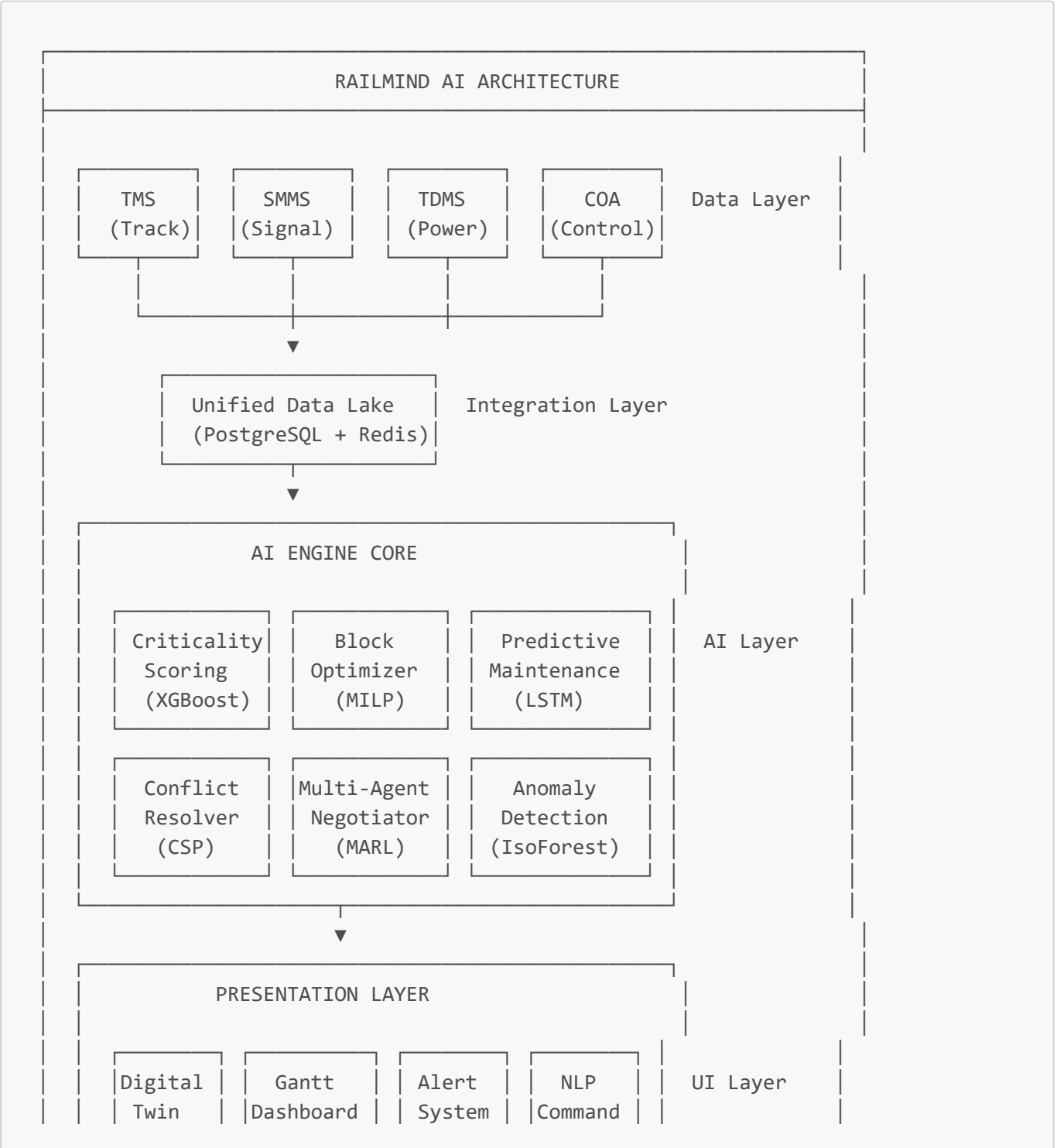
1.4 Core Technical Strategy

AI/ML Stack (Real, Not Buzzword)

Component	Algorithm/Technique	Purpose
Block Optimization	Mixed-Integer Linear Programming (MILP) using Google OR-Tools	Find the mathematically optimal block schedule
Multi-Dept Coordination	Multi-Agent Reinforcement Learning (MARL)	Simulate department negotiation
Priority Scoring	Gradient-Boosted Decision Trees (XGBoost)	Rank maintenance tasks by criticality

Component	Algorithm/Technique	Purpose
Predictive Maintenance	LSTM Time-Series Forecasting	Predict when assets will need maintenance
Conflict Detection	Constraint Satisfaction Problem (CSP) Solver	Identify scheduling conflicts instantly
NLP Commands	Fine-tuned LLM with Railway Domain	Natural language interface for controllers
Anomaly Detection	Isolation Forest / Autoencoders	Detect unusual asset behavior

Architecture Overview



```
│  │  │  Map   │ │  View    │ │        │ │Interface│ │              │
│  │  └────────┘ └──────────┘ └────────┘ └─────────┘ │              │
│  └──────────────────────────────────────────────────┘              │
└──────────────────────────────────────────────────────────────────┘
```

Tech Stack

Layer	Technology	Reason
Frontend	React.js + Leaflet.js/Mapbox + D3.js	Interactive maps, Gantt charts, real-time dashboards
Backend	FastAPI (Python)	Async, fast, AI-native, great for ML serving
Database	PostgreSQL + TimescaleDB	Time-series maintenance data + relational data
Cache	Redis	Real-time block status caching
AI/ML	Python (scikit-learn, XGBoost, PyTorch, OR-Tools)	Core optimization and ML
Real-time	WebSockets (Socket.IO)	Live updates on block status
Deployment	Docker + Docker Compose	Easy demo deployment

1.5 Detailed Prototype Architecture — The Full Blueprint

Complete Project Folder Structure

```
railmind-ai/
│
├── docker-compose.yml              # One-command deployment
├── .env.example                    # Environment variables template
├── README.md                       # Project documentation
│
├── backend/                        # FastAPI Backend
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── main.py                     # FastAPI app entry point
│   ├── config.py                   # Settings & environment config
│   │
│   ├── api/                        # API Routes
│   │   ├── __init__.py
│   │   ├── routes/
│   │   │   ├── maintenance.py      # CRUD for maintenance tasks (TMS/SMMS/TDMS)
│   │   │   ├── timetable.py        # Train timetable & COA corridor data
│   │   │   ├── blocks.py           # Block planning CRUD & generation
│   │   │   ├── optimizer.py        # Trigger AI optimization endpoints
│   │   │   ├── analytics.py        # Dashboard metrics & KPIs
│   │   │   ├── notifications.py    # Alert & notification endpoints
│   │   │   └── nlp.py              # NLP command processing
```

```
│   │   └── websockets/
│   │       └── live_updates.py     # Real-time block status via WebSocket
│   │
│   ├── models/                     # SQLAlchemy ORM Models
│   │   ├── __init__.py
│   │   ├── maintenance_task.py     # Unified task model (TMS+SMMS+TDMS)
│   │   ├── train_schedule.py       # Train timetable model
│   │   ├── block_plan.py           # Generated block plan model
│   │   ├── corridor.py             # Corridor & section model
│   │   ├── asset.py                # Railway asset model (track/signal/OHE)
│   │   └── user.py                 # User roles (Controller/PWI/DEN)
│   │
│   ├── services/                   # Business Logic Layer
│   │   ├── __init__.py
│   │   ├── data_integrator.py      # Merges TMS+SMMS+TDMS+COA data
│   │   ├── criticality_scorer.py   # XGBoost criticality scoring
│   │   ├── block_optimizer.py      # MILP optimizer (Google OR-Tools)
│   │   ├── conflict_resolver.py    # CSP-based conflict detection
│   │   ├── predictive_engine.py    # LSTM predictive maintenance
│   │   ├── multi_agent.py          # Multi-Agent department negotiation
│   │   ├── anomaly_detector.py     # Isolation Forest anomaly detection
│   │   ├── nlp_processor.py        # NLP command parser
│   │   └── notification_service.py # SMS/Email/Dashboard alerts
│   │
│   ├── ai_models/                  # Trained ML Models (pickle/pt files)
│   │   ├── criticality_model.pkl   # Pre-trained XGBoost
│   │   ├── lstm_predictor.pt       # Pre-trained LSTM
│   │   └── anomaly_model.pkl       # Pre-trained Isolation Forest
│   │
│   ├── data/                       # Synthetic Data & Seeders
│   │   ├── seed_maintenance.py     # Generate 200+ realistic maintenance tasks
│   │   ├── seed_timetable.py       # Generate 150+ train schedules
│   │   ├── seed_corridors.py       # Generate real IR section data
│   │   ├── seed_assets.py          # Generate track/signal/OHE assets
│   │   └── sample_data/
│   │       ├── tms_defects.json    # Sample TMS data
│   │       ├── smms_signals.json   # Sample SMMS data
│   │       ├── tdms_ohe.json       # Sample TDMS data
│   │       ├── coa_timetable.json  # Sample COA timetable
│   │       └── sections.json       # Real IR section/KM data
│   │
│   └── tests/                      # Unit + Integration Tests
│       ├── test_optimizer.py
│       ├── test_criticality.py
│       ├── test_conflict.py
│       └── test_api.py
│
├── frontend/                       # React.js Frontend
│   ├── Dockerfile
│   ├── package.json
│   ├── public/
│   │   ├── index.html
│   │   └── favicon.ico             # RailMind AI logo
│   │
```

```
│   └── src/
│       ├── App.jsx                 # Main app with routing
│       ├── index.css               # Global styles + design tokens
│       │
│       ├── components/
│       │   ├── layout/
│       │   │   ├── Sidebar.jsx         # Navigation sidebar
│       │   │   ├── Header.jsx          # Top bar with alerts count
│       │   │   └── DashboardLayout.jsx # Main layout wrapper
│       │   │
│       │   ├── map/
│       │   │   ├── DigitalTwinMap.jsx   # Leaflet map with railway overlay
│       │   │   ├── TrainMarker.jsx      # Moving train icon on map
│       │   │   ├── SectionOverlay.jsx   # Color-coded track sections
│       │   │   ├── BlockOverlay.jsx     # Block window shading on map
│       │   │   └── AssetPopup.jsx       # Click popup for asset details
│       │   │
│       │   ├── scheduler/
│       │   │   ├── GanttChart.jsx       # Interactive Gantt chart (D3.js)
│       │   │   ├── BlockCard.jsx        # Individual block request card
│       │   │   ├── ConflictPanel.jsx    # Conflict visualization panel
│       │   │   ├── WeeklyPlanner.jsx    # Weekly block plan view
│       │   │   └── MonthlyPlanner.jsx   # Monthly block plan view
│       │   │
│       │   ├── ai/
│       │   │   ├── OptimizerPanel.jsx   # "Generate Plan" button + results
│       │   │   ├── CriticalityTable.jsx # Ranked task list by priority
│       │   │   ├── PredictionChart.jsx  # Asset degradation curves
│       │   │   ├── AgentNegotiation.jsx # Multi-agent visualization
│       │   │   └── NLPCommandBar.jsx    # Natural language input bar
│       │   │
│       │   ├── analytics/
│       │   │   ├── KPICards.jsx         # Big number metric cards
│       │   │   ├── BeforeAfterChart.jsx # Optimization comparison chart
│       │   │   ├── UtilizationGauge.jsx # Block utilization gauge
│       │   │   └── TrendChart.jsx       # Historical trends line chart
│       │   │
│       │   └── common/
│       │       ├── AlertBanner.jsx      # Emergency alert banner
│       │       ├── LoadingSpinner.jsx   # AI processing indicator
│       │       └── ExportButton.jsx     # PDF/CSV export
│       │
│       ├── pages/
│       │   ├── Dashboard.jsx        # Main overview page
│       │   ├── DigitalTwin.jsx      # Full-screen Digital Twin page
│       │   ├── BlockPlanner.jsx     # Block scheduling page
│       │   ├── Maintenance.jsx      # Task management page
│       │   ├── Analytics.jsx        # KPI & reports page
│       │   └── Settings.jsx         # Configuration page
│       │
│       ├── hooks/
│       │   ├── useWebSocket.js      # Real-time updates hook
│       │   ├── useOptimizer.js      # AI optimization trigger hook
│       │   └── useMapData.js        # Map data fetching hook
```

```
│       │
│       ├── services/
│       │   └── api.js               # Axios API client
│       │
│       └── utils/
│           ├── constants.js         # Railway constants (colors, codes)
│           ├── formatters.js        # Date/time/KM formatters
│           └── railwayData.js       # Station codes, section mapping
│
├── ml_notebooks/                    # Jupyter Notebooks for AI development
│   ├── 01_data_exploration.ipynb
│   ├── 02_criticality_model.ipynb   # Train XGBoost model
│   ├── 03_lstm_predictor.ipynb      # Train LSTM model
│   ├── 04_optimizer_testing.ipynb   # Test MILP optimizer
│   └── 05_multi_agent_sim.ipynb     # Multi-Agent simulation
│
└── docs/
    ├── API_DOCS.md                  # API documentation
    ├── ARCHITECTURE.md              # Detailed architecture doc
    └── DEMO_SCRIPT.md               # Step-by-step demo script
```

Database Schema (PostgreSQL)

```sql
-- ═══════════════════════════════════════════════════════════════
-- TABLE 1: railway_sections — The physical railway infrastructure
-- ═══════════════════════════════════════════════════════════════
CREATE TABLE railway_sections (
    id              SERIAL PRIMARY KEY,
    section_code    VARCHAR(20) UNIQUE NOT NULL,  -- e.g., "CSMT-KYN"
    section_name    VARCHAR(100) NOT NULL,         -- e.g., "CSMT to Kalyan"
    division        VARCHAR(50) NOT NULL,          -- e.g., "Mumbai Central"
    zone            VARCHAR(50) NOT NULL,          -- e.g., "Central Railway"
    km_start        DECIMAL(8,2) NOT NULL,         -- e.g., 0.00
    km_end          DECIMAL(8,2) NOT NULL,         -- e.g., 54.00
    track_type      VARCHAR(20),                   -- SGL/DBL/TRIPLE/QUAD
    traction        VARCHAR(20),                   -- AC25KV/DC1500V/DIESEL
    max_speed_kmph  INT DEFAULT 110,
    trains_per_day  INT,                           -- traffic density
    is_ghat_section BOOLEAN DEFAULT FALSE,
    created_at      TIMESTAMP DEFAULT NOW()
);

-- ═══════════════════════════════════════════════════════════════
-- TABLE 2: railway_assets — Track, Signal, OHE equipment inventory
-- ═══════════════════════════════════════════════════════════════
CREATE TABLE railway_assets (
    id              SERIAL PRIMARY KEY,
    asset_code      VARCHAR(30) UNIQUE NOT NULL,   -- e.g., "TRK-KYN-045"
    asset_type      VARCHAR(20) NOT NULL,          -- TRACK/SIGNAL/OHE/BRIDGE
    department      VARCHAR(20) NOT NULL,          -- ENGG/SNT/TRD
    section_id      INT REFERENCES railway_sections(id),
```

```
    km_location     DECIMAL(8,2) NOT NULL,
    km_end          DECIMAL(8,2),                  -- for range assets
    equipment_name  VARCHAR(100),                  -- e.g., "Color Light Signal"
    install_date    DATE,
    last_maintained DATE,
    health_score    DECIMAL(5,2) DEFAULT 100.00,   -- 0-100 health %
    status          VARCHAR(20) DEFAULT 'ACTIVE',  -- ACTIVE/DEGRADED/FAILED
    metadata        JSONB                          -- flexible extra fields
);

-- ═══════════════════════════════════════════════════════════════
-- TABLE 3: maintenance_tasks — Unified defects + overdue tasks
--          (Merges TMS + SMMS + TDMS data)
-- ═══════════════════════════════════════════════════════════════
CREATE TABLE maintenance_tasks (
    id                  SERIAL PRIMARY KEY,
    task_code           VARCHAR(30) UNIQUE NOT NULL,    -- "TMS-MUM-2026-04523"
    source_system       VARCHAR(10) NOT NULL,           -- TMS/SMMS/TDMS
    department          VARCHAR(20) NOT NULL,           -- ENGG/SNT/TRD
    asset_id            INT REFERENCES railway_assets(id),
    section_id          INT REFERENCES railway_sections(id),
    km_from             DECIMAL(8,2) NOT NULL,
    km_to               DECIMAL(8,2) NOT NULL,
    task_type           VARCHAR(50) NOT NULL,           -- "Rail Fracture Repair"
    severity            VARCHAR(20) NOT NULL,           --
CRITICAL/HIGH/MEDIUM/LOW
    detected_date       DATE NOT NULL,
    due_date            DATE NOT NULL,
    estimated_hours     DECIMAL(4,1) NOT NULL,          -- block hours needed
    block_type_required VARCHAR(30) NOT NULL,           --
TRAFFIC/POWER/DISCONNECTION
    line_affected       VARCHAR(10),                    -- UP/DN/BOTH
    speed_restriction   INT,                            -- if imposed, in kmph
    is_overdue          BOOLEAN DEFAULT FALSE,
    criticality_score   DECIMAL(5,2),                   -- AI-generated (0-100)
    predicted_failure   DATE,                           -- LSTM prediction
    status              VARCHAR(20) DEFAULT 'PENDING',  -- PENDING/SCHEDULED/DONE
    created_at          TIMESTAMP DEFAULT NOW()
);

-- ═══════════════════════════════════════════════════════════════
-- TABLE 4: train_schedules — Train timetable from COA
-- ═══════════════════════════════════════════════════════════════
CREATE TABLE train_schedules (
    id              SERIAL PRIMARY KEY,
    train_no        VARCHAR(10) NOT NULL,           -- "12124"
    train_name      VARCHAR(100) NOT NULL,          -- "Deccan Queen"
    train_type      VARCHAR(20) NOT NULL,           --
RAJDHANI/SF/EXPRESS/PASS/GOODS
    priority        INT NOT NULL,                   -- 1=highest (Rajdhani) to
5=lowest
    section_id      INT REFERENCES railway_sections(id),
    direction       VARCHAR(5) NOT NULL,            -- UP/DN
    entry_km        DECIMAL(8,2) NOT NULL,
```

```
    exit_km         DECIMAL(8,2) NOT NULL,
    entry_time      TIME NOT NULL,
    exit_time       TIME NOT NULL,
    days_of_run     VARCHAR(20) DEFAULT 'DAILY',    -- DAILY/MWF/TTSS/etc
    is_special      BOOLEAN DEFAULT FALSE,          -- festival specials
    avg_delay_min   INT DEFAULT 0
);

-- ═══════════════════════════════════════════════════════════
-- TABLE 5: block_plans — AI-generated optimized block schedules
-- ═══════════════════════════════════════════════════════════
CREATE TABLE block_plans (
    id              SERIAL PRIMARY KEY,
    plan_code       VARCHAR(30) UNIQUE NOT NULL,    -- "BLK-MUM-W35-001"
    plan_type       VARCHAR(20) NOT NULL,           -- WEEKLY/MONTHLY
    section_id      INT REFERENCES railway_sections(id),
    generated_at    TIMESTAMP DEFAULT NOW(),
    generated_by    VARCHAR(20) DEFAULT 'AI',       -- AI/MANUAL
    plan_start_date DATE NOT NULL,
    plan_end_date   DATE NOT NULL,
    total_blocks    INT,
    total_block_hrs DECIMAL(6,1),
    utilization_pct DECIMAL(5,2),                   -- block utilization %
    optimization_score DECIMAL(5,2),                -- optimizer objective value
    status          VARCHAR(20) DEFAULT 'DRAFT',    -- DRAFT/APPROVED/ACTIVE/DONE
    approved_by     VARCHAR(50),                    -- Controller name/ID
    approved_at     TIMESTAMP
);

-- ═══════════════════════════════════════════════════════════
-- TABLE 6: block_windows — Individual blocks within a plan
-- ═══════════════════════════════════════════════════════════
CREATE TABLE block_windows (
    id              SERIAL PRIMARY KEY,
    plan_id         INT REFERENCES block_plans(id),
    block_code      VARCHAR(30) UNIQUE NOT NULL,    -- "BW-MUM-W35-003"
    block_date      DATE NOT NULL,
    start_time      TIME NOT NULL,
    end_time        TIME NOT NULL,
    duration_hrs    DECIMAL(4,1) NOT NULL,
    section_id      INT REFERENCES railway_sections(id),
    km_from         DECIMAL(8,2) NOT NULL,
    km_to           DECIMAL(8,2) NOT NULL,
    block_type      VARCHAR(30) NOT NULL,           --
TRAFFIC/POWER/INTEGRATED/MEGA
    line_blocked    VARCHAR(10) NOT NULL,            -- UP/DN/BOTH
    is_integrated   BOOLEAN DEFAULT FALSE,          -- multi-dept integrated
block?
    departments     VARCHAR(50)[],                  -- {ENGG,SNT,TRD}
    status          VARCHAR(20) DEFAULT 'PLANNED'   --
PLANNED/ACTIVE/COMPLETED/CANCELLED
);

-- ═══════════════════════════════════════════════════════════
```

```sql
-- TABLE 7: block_task_assignments — Links tasks to block windows
-- ═══════════════════════════════════════════════════════════════
CREATE TABLE block_task_assignments (
    id              SERIAL PRIMARY KEY,
    block_id        INT REFERENCES block_windows(id),
    task_id         INT REFERENCES maintenance_tasks(id),
    sequence_order  INT,                          -- work sequence within block
    start_offset_min INT,                         -- minutes from block start
    duration_min    INT,                          -- task duration in minutes
    assigned_team   VARCHAR(50),                  -- "SSE/P.Way/Kalyan"
    status          VARCHAR(20) DEFAULT 'ASSIGNED'
);

-- ═══════════════════════════════════════════════════════════════
-- TABLE 8: conflict_log — AI conflict detection & resolution log
-- ═══════════════════════════════════════════════════════════════
CREATE TABLE conflict_log (
    id              SERIAL PRIMARY KEY,
    detected_at     TIMESTAMP DEFAULT NOW(),
    conflict_type   VARCHAR(30) NOT NULL,         -- 
TRAIN_CLASH/DEPT_OVERLAP/RESOURCE
    description     TEXT NOT NULL,
    block_id_1      INT REFERENCES block_windows(id),
    block_id_2      INT,
    train_id        INT REFERENCES train_schedules(id),
    resolution      TEXT,                         -- AI's proposed resolution
    resolution_type VARCHAR(20),                  -- 
RESCHEDULE/MERGE/SPLIT/CANCEL
    resolved        BOOLEAN DEFAULT FALSE,
    resolved_at     TIMESTAMP
);
```

API Endpoint Map (FastAPI)

```
┌──────────────────────────────────────────────────────────────────┐
│                        API ENDPOINT MAP                          │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  📁 /api/v1/maintenance                                          │
│  ├── GET    /tasks              → List all maintenance tasks     │
│  ├── GET    /tasks/{id}         → Get task details               │
│  ├── POST   /tasks              → Create new maintenance task    │
│  ├── GET    /tasks/critical     → Get tasks sorted by criticality│
│  ├── GET    /tasks/overdue      → Get overdue tasks only         │
│  └── GET    /tasks/by-section/{section_id}  → Tasks by section   │
│                                                                  │
│  📁 /api/v1/timetable                                            │
│  ├── GET    /trains             → List all train schedules       │
│  ├── GET    /trains/by-section/{section_id}  → Trains on a section│
│  └── GET    /corridor/{section_id}/availability → Free windows today│
│                                                                  │
```

```
│  📁 /api/v1/optimizer                                                    │
│  ├── POST   /generate-plan      → ⭐ Trigger AI block plan generation    │
│  ├── POST   /re-optimize        → Re-optimize after changes              │
│  ├── POST   /what-if            → What-if scenario simulation            │
│  └── GET    /explain/{plan_id}  → Explainability: why this plan?         │
│                                                                          │
│  📁 /api/v1/blocks                                                       │
│  ├── GET    /plans              → List all block plans                   │
│  ├── GET    /plans/{id}         → Get plan with all block windows        │
│  ├── PUT    /plans/{id}/approve → Controller approves a plan             │
│  └── GET    /plans/{id}/export  → Export plan as PDF                     │
│                                                                          │
│  📁 /api/v1/analytics                                                    │
│  ├── GET    /kpis               → Dashboard KPI metrics                  │
│  └── GET    /comparison         → Before vs After optimization stats     │
│                                                                          │
│  📁 /api/v1/nlp                                                          │
│  └── POST   /command            → Process NLP text command               │
│                                                                          │
│  🔌 WebSocket /ws/live-updates  → Real-time block status stream          │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

Data Flow Pipeline — How Every Request Travels

```
USER REQUEST: "Generate optimal weekly block plan for CSMT-Kalyan"
    │
    ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ STEP 1: DATA AGGREGATION  (data_integrator.py)                           │
│                                                                          │
│  Fetch from DB:                                                          │
│  ├── 47 maintenance tasks (TMS: 22, SMMS: 14, TDMS: 11)                  │
│  ├── 156 train schedules for the section                                 │
│  ├── Corridor data: daily block windows (10:00-13:00 typical)            │
│  └── Asset health scores for 340 assets                                  │
│                                                                          │
│  Output: Unified task list + constraint matrix                           │
└──────────────────────┬───────────────────────────────────────────────────┘
                       ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ STEP 2: CRITICALITY SCORING  (criticality_scorer.py)                     │
│                                                                          │
│  For each of 47 tasks, compute:                                          │
│                                                                          │
│  criticality_score = XGBoost.predict([                                   │
│      severity_encoded,        # CRITICAL=4, HIGH=3, MED=2, LOW=1         │
│      days_overdue,            # negative = days until due                │
│      traffic_density,         # trains/day on affected section           │
│      safety_impact_score,     # 1-10 based on defect type                │
│      cascading_risk,          # will failure affect other assets?        │
```

```
│      speed_restriction_impact,# revenue loss from speed limit    │
│      historical_failure_rate, # past failures at this location   │
│      asset_remaining_life,    # % life remaining                 │
│      weather_risk_factor,     # monsoon/flood risk               │
│      last_maintenance_gap     # days since last maintenance      │
│  ])                                                              │
│                                                                  │
│  Output: 47 tasks ranked by criticality (0-100 score)            │
└──────────────────────────┬───────────────────────────────────────┘
                           ▼
┌──────────────────────────────────────────────────────────────────┐
│ STEP 3: CONFLICT DETECTION  (conflict_resolver.py)               │
│                                                                  │
│  For each task, check:                                           │
│  ├── Train conflicts: Does any train pass during requested time? │
│  ├── Dept conflicts: Does another dept need same section?        │
│  ├── Resource conflicts: Is required equipment available?        │
│  └── Adjacent conflicts: Blocks too close on same corridor?      │
│                                                                  │
│  Output: Conflict matrix (47 x 47) + train conflict windows      │
└──────────────────────────┬───────────────────────────────────────┘
                           ▼
┌──────────────────────────────────────────────────────────────────┐
│ STEP 4: MULTI-AGENT NEGOTIATION  (multi_agent.py)                │
│                                                                  │
│  Create 3 AI agents:                                             │
│  ├── ENGG_Agent: Advocates for 22 Engineering tasks              │
│  ├── SNT_Agent:  Advocates for 14 S&T tasks                      │
│  └── TRD_Agent:  Advocates for 11 TRD tasks                      │
│                                                                  │
│  Negotiation rounds:                                             │
│  ├── Round 1: Each agent submits preferred time slots            │
│  ├── Round 2: Identify overlaps → propose integrated blocks      │
│  ├── Round 3: Resolve remaining conflicts via priority bidding   │
│  └── Round 4: Arbitrator finds Nash Equilibrium allocation       │
│                                                                  │
│  Output: Negotiated task-slot preferences per department         │
└──────────────────────────┬───────────────────────────────────────┘
                           ▼
┌──────────────────────────────────────────────────────────────────┐
│ STEP 5: MILP OPTIMIZATION  (block_optimizer.py)                  │
│                                                                  │
│  Solver: Google OR-Tools CP-SAT / MILP                           │
│                                                                  │
│  Decision Variables:                                             │
│    x[i][d][s] = 1 if task i assigned to day d, slot s            │
│    merge[i][j] = 1 if tasks i and j share a block                │
│                                                                  │
│  Objective: MINIMIZE                                             │
│    total_block_time                                              │
│    + α × critical_task_delay_penalty                             │
│    - β × integrated_block_bonus                                  │
│    + γ × train_disruption_penalty                                │
│                                                                  │
```

```
│  Constraints: (12 hard constraints)                                  │
│    C1:  No block overlaps with train paths                           │
│    C2:  Critical tasks within deadline                               │
│    C3:  Block within corridor availability window                    │
│    C4:  Max 2 blocks per section per day                             │
│    C5:  Minimum gap between consecutive blocks (30 min)              │
│    C6:  Power block required before TRD work                         │
│    C7:  Disconnection required before S&T work                       │
│    C8:  Block duration >= task estimated time + buffer               │
│    C9:  Resource availability (tamping machine, OHE car)             │
│    C10: Maximum 1 mega block per week per section                    │
│    C11: Integrated block only if departments agree (from Step 4)     │
│    C12: Safety blocks cannot be deferred beyond due date             │
│                                                                      │
│  Solve time target: < 5 seconds for division-level problem           │
│                                                                      │
│  Output: Optimal block plan (31 blocks from 47 tasks)                │
└──────────────────────────┬───────────────────────────────────────────┘
                           ▼
┌──────────────────────────────────────────────────────────────────────┐
│ STEP 6: PLAN ASSEMBLY & EXPLAINABILITY                               │
│                                                                      │
│  ├── Assemble block_windows with task assignments                    │
│  ├── Generate Gantt chart data (JSON for D3.js)                      │
│  ├── Generate map overlay data (GeoJSON for Leaflet)                 │
│  ├── Compute KPIs: utilization%, time saved, conflicts resolved      │
│  ├── Generate natural language explanations for each decision        │
│  │   Example: "Tasks T-012 (Engg) and T-031 (S&T) merged into        │
│  │   integrated block BW-035 because they overlap at KM 52-54        │
│  │   and both fit within the 10:00-13:00 corridor window."           │
│  └── Store plan in DB with status = DRAFT                            │
│                                                                      │
│  Output: Complete block plan + visualization data + explanations     │
└──────────────────────────┬───────────────────────────────────────────┘
                           ▼
┌──────────────────────────────────────────────────────────────────────┐
│ STEP 7: PRESENTATION TO CONTROLLER                                   │
│                                                                      │
│  Dashboard shows:                                                    │
│  ├── 📊 Gantt chart: Drag-drop to manually adjust if needed          │
│  ├── 🗺️ Digital Twin: Blocks overlaid on railway map                 │
│  ├── 📈 KPI cards: "34% fewer blocks | 89% utilization | 0 conflicts" │
│  ├── 📋 Task list: Sorted by criticality with status badges          │
│  ├── ⚠️ Conflict log: Any unresolved issues flagged                  │
│  └── ✅ Approve button: Controller signs off → status = APPROVED     │
│                                                                      │
│  WebSocket pushes live updates to all connected dashboards           │
└──────────────────────────────────────────────────────────────────────┘
```

Frontend Component Architecture (React)

```
App.jsx
├── <BrowserRouter>
│   ├── <DashboardLayout>
│   │   ├── <Sidebar>                    # Navigation
│   │   │   ├── Dashboard
│   │   │   ├── Digital Twin
│   │   │   ├── Block Planner
│   │   │   ├── Maintenance Tasks
│   │   │   ├── Analytics
│   │   │   └── Settings
│   │   │
│   │   ├── <Header>                     # Top bar
│   │   │   ├── <NLPCommandBar />        # "Ask RailMind..."
│   │   │   ├── <AlertBadge count={3} /> # Active alerts
│   │   │   └── <UserProfile />
│   │   │
│   │   └── <Routes>
│   │       │
│   │       ├── "/" → <Dashboard>                    # ⭐ MAIN PAGE
│   │       │   ├── <KPICards>                       # 4 big metric cards
│   │       │   │   ├── Asset Availability: 94.2%
│   │       │   │   ├── Block Utilization: 89%
│   │       │   │   ├── Tasks Pending: 47
│   │       │   │   └── Conflicts: 0
│   │       │   ├── <MiniMap>                        # Small overview map
│   │       │   ├── <UpcomingBlocks>                 # Next 5 blocks list
│   │       │   ├── <CriticalAlerts>                 # Red/amber alerts
│   │       │   └── <TrendChart>                     # 7-day trend line
│   │       │
│   │       ├── "/twin" → <DigitalTwin>              # ⭐ WOW PAGE
│   │       │   ├── <DigitalTwinMap>                 # Full-screen Leaflet
│   │       │   │   ├── <TileLayer> (OpenStreetMap)
│   │       │   │   ├── <RailwayTrackLayer>          # GeoJSON track lines
│   │       │   │   ├── <SectionOverlay>             # Color: health score
│   │       │   │   │   ├── Green: > 80% health
│   │       │   │   │   ├── Yellow: 50-80% health
│   │       │   │   │   └── Red: < 50% health
│   │       │   │   ├── <TrainMarker> × N            # Moving train dots
│   │       │   │   ├── <BlockOverlay>               # Shaded block zones
│   │       │   │   └── <DefectMarker> × N           # Defect pin icons
│   │       │   ├── <MapControls>                    # Layer toggles
│   │       │   │   ├── Toggle: Track Health
│   │       │   │   ├── Toggle: Active Blocks
│   │       │   │   ├── Toggle: Train Positions
│   │       │   │   └── Toggle: Defect Markers
│   │       │   └── <AssetDetailPanel>               # Slide-out detail
│   │       │
│   │       ├── "/planner" → <BlockPlanner>          # ⭐ CORE FEATURE
│   │       │   ├── <PlannerToolbar>
│   │       │   │   ├── [Generate Weekly Plan] button
│   │       │   │   ├── [Generate Monthly Plan] button
│   │       │   │   ├── Section dropdown selector
```

```
│   │       │   │   └── Date range picker
│   │       │   ├── <GanttChart>                    # D3.js Gantt
│   │       │   │   ├── Y-axis: Sections/KM ranges
│   │       │   │   ├── X-axis: Time slots (hours)
│   │       │   │   ├── Blocks as colored bars
│   │       │   │   │   ├── Blue: Engineering
│   │       │   │   │   ├── Orange: S&T
│   │       │   │   │   ├── Purple: TRD
│   │       │   │   │   └── Green: Integrated
│   │       │   │   ├── Train paths as thin lines
│   │       │   │   └── Drag-drop to adjust blocks
│   │       │   ├── <ConflictPanel>                 # Right sidebar
│   │       │   │   ├── Conflict description
│   │       │   │   ├── Affected trains list
│   │       │   │   └── AI suggested resolution
│   │       │   └── <ApprovalPanel>
│   │       │       ├── Optimization summary
│   │       │       ├── [Approve Plan] button
│   │       │       └── [Export PDF] button
│   │       │
│   │       ├── "/maintenance" → <Maintenance>
│   │       │   ├── <FilterBar> (dept, severity, status)
│   │       │   ├── <CriticalityTable>              # Sortable task table
│   │       │   ├── <PredictionChart>               # LSTM curves
│   │       │   └── <AgentNegotiation>              # Multi-agent viz
│   │       │
│   │       └── "/analytics" → <Analytics>
│   │           ├── <BeforeAfterChart>              # Split comparison
│   │           ├── <UtilizationGauge>              # Donut gauge
│   │           ├── <DepartmentBreakdown>           # Pie chart
│   │           └── <HistoricalTrend>               # Line chart
```

Docker Compose Configuration

```yaml
# docker-compose.yml — ONE COMMAND TO RULE THEM ALL
version: '3.8'

services:
  # ─── PostgreSQL Database ───
  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: railmind
      POSTGRES_USER: railmind_user
      POSTGRES_PASSWORD: railmind_pass_2026
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./backend/data/init.sql:/docker-entrypoint-initdb.d/init.sql
    healthcheck:
```

```
      test: ["CMD-SHELL", "pg_isready -U railmind_user"]
      interval: 5s
      timeout: 5s
      retries: 5

  # ── Redis Cache ──
  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s

  # ── FastAPI Backend ──
  backend:
    build: ./backend
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: postgresql://railmind_user:railmind_pass_2026@db:5432/railmind
      REDIS_URL: redis://redis:6379/0
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_healthy
    volumes:
      - ./backend:/app
    command: uvicorn main:app --host 0.0.0.0 --port 8000 --reload

  # ── React Frontend ──
  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    environment:
      REACT_APP_API_URL: http://localhost:8000/api/v1
      REACT_APP_WS_URL: ws://localhost:8000/ws
    depends_on:
      - backend
    volumes:
      - ./frontend/src:/app/src
    command: npm start

volumes:
  pgdata:

# USAGE: docker-compose up --build
# Then open: http://localhost:3000
```

1.6 Detailed Prototype Build Roadmap — Step by Step

> This is the EXACT sequence to follow. Each step has dependencies — don't skip ahead.

🔷 PHASE 1: Project Scaffolding (Hours 0-1)

Goal: Repository and infrastructure ready to go.

```
Tasks:
├── Initialize Git repo with .gitignore
├── Create folder structure (as shown above)
├── Write docker-compose.yml
├── Create backend/Dockerfile (Python 3.11 + FastAPI)
├── Create frontend/Dockerfile (Node 18 + React)
├── Write backend/requirements.txt
├── Write frontend/package.json with all dependencies
├── Create .env.example
└── Run: docker-compose up --build (verify everything starts)
```

Key Dependencies to Install:

```
# backend/requirements.txt
fastapi==0.104.1
uvicorn==0.24.0
sqlalchemy==2.0.23
asyncpg==0.29.0
psycopg2-binary==2.9.9
alembic==1.13.0
redis==5.0.1
ortools==9.8.3296
scikit-learn==1.3.2
xgboost==2.0.2
pandas==2.1.4
numpy==1.26.2
python-socketio==5.10.0
pydantic==2.5.2
httpx==0.25.2
python-jose==3.3.0
reportlab==4.0.8

# frontend/package.json (key deps)
react: ^18.2.0
react-router-dom: ^6.20.0
leaflet: ^1.9.4
react-leaflet: ^4.2.1
d3: ^7.8.5
axios: ^1.6.2
socket.io-client: ^4.7.2
@tanstack/react-query: ^5.12.0
recharts: ^2.10.3
lucide-react: ^0.294.0
```

```
date-fns: ^2.30.0
framer-motion: ^10.16.0
```

Deliverables: ✅ `docker-compose up` works. Empty app loads at localhost:3000.

🔷 PHASE 2: Database & Data Models (Hours 1-3)

Goal: Database schema created and seeded with realistic synthetic data.

```
Tasks:
├── Write SQL schema (8 tables as shown above)
├── Create SQLAlchemy ORM models
├── Build data seeder scripts:
│   ├── seed_corridors.py    → 5 sections (CSMT-KYN, KYN-KSR, etc.)
│   ├── seed_assets.py       → 340 assets across sections
│   ├── seed_maintenance.py  → 200+ tasks (realistic defect types)
│   ├── seed_timetable.py    → 156 trains with real names/numbers
│   └── seed_all.py          → Master seeder runs everything
├── Run seeders and verify data in DB
└── Create basic CRUD API endpoints for all tables
```

Pro Tip: Use REAL Indian Railways data:

- Real train numbers (12124 Deccan Queen, 12951 Mumbai Rajdhani)
- Real station codes (CSMT, KYN, KSR, TNA, IGP)
- Real KM markers from timetable books
- Real defect types from IRPWM (Rail Fracture, Gauge Face Wear, Fish Plate Crack)

Deliverables: ✅ Database populated. `GET /api/v1/maintenance/tasks` returns 200+ tasks.

🔷 PHASE 3: Criticality Scoring Engine (Hours 3-6)

Goal: AI model that ranks every maintenance task by urgency and impact.

```
Tasks:
├── Generate training data (2000+ labeled samples)
│   Labels: Manual criticality scores based on IR maintenance rules
├── Feature engineering:
│   ├── severity_encoded (CRITICAL=4, HIGH=3, MEDIUM=2, LOW=1)
│   ├── days_until_due (negative = overdue)
│   ├── traffic_density (trains/day on section)
│   ├── safety_impact (1-10 by defect type lookup)
│   ├── cascading_risk (does failure affect signals/power?)
│   ├── speed_restriction_impact (revenue loss estimate)
│   ├── historical_failure_rate (past failures at location)
│   ├── asset_remaining_life_pct
│   ├── weather_risk (monsoon factor for location)
```

```
│   └── maintenance_gap_days
├── Train XGBoost model in Jupyter notebook
├── Evaluate: target AUC > 0.90
├── Export model as .pkl file
├── Create criticality_scorer.py service
│   └── score_all_tasks() → returns ranked list
├── Create API: GET /api/v1/maintenance/tasks/critical
└── Build <CriticalityTable> React component
```

Deliverables: ✅ Every task has a 0-100 criticality score. Table sorts by urgency.

🔷 PHASE 4: MILP Block Optimizer — The Core Brain (Hours 6-12)

Goal: The mathematical optimizer that generates optimal block plans.

```
Tasks:
├── Define MILP model in block_optimizer.py:
│   ├── Decision variables: x[task][day][slot] ∈ {0,1}
│   ├── Objective function (minimize total downtime)
│   ├── 12 hard constraints (train paths, deadlines, etc.)
│   └── 4 soft constraints (preferences, efficiency bonuses)
│
├── Implement with Google OR-Tools CP-SAT solver
│
├── Build the core function:
│   def generate_optimal_plan(
│       section_id: int,
│       plan_type: "WEEKLY" | "MONTHLY",
│       tasks: List[MaintenanceTask],
│       trains: List[TrainSchedule],
│       corridor_windows: List[TimeWindow]
│   ) -> BlockPlan
│
├── Test with sample data:
│   ├── Input: 47 tasks + 156 trains + corridor windows
│   ├── Verify: No constraint violations
│   ├── Measure: Solve time < 5 seconds
│   └── Compare: Manual plan vs AI plan (show improvement)
│
├── Create API: POST /api/v1/optimizer/generate-plan
│   ├── Request: { section_id, plan_type, date_range }
│   ├── Response: { plan with block_windows, KPIs, explanation }
│   └── Show progress via WebSocket during optimization
│
└── Build <OptimizerPanel> React component
    ├── "Generate Plan" button with loading animation
    ├── Progress bar during optimization
    └── Results summary with KPI highlights
```

This is the MOST CRITICAL phase. Spend extra time here. A working optimizer is the difference between winning and losing.

Deliverables: ✅ Click "Generate Plan" → 31 optimized blocks appear in < 5 seconds.

🔷 PHASE 5: Interactive Gantt Chart (Hours 12-16)

Goal: Visual block schedule that judges can see and interact with.

```
Tasks:
├── Build GanttChart.jsx using D3.js:
│   ├── Y-axis: Railway sections / KM ranges
│   ├── X-axis: Time (hours within a day, days within a week)
│   ├── Blocks as colored horizontal bars
│   │   ├── Engineering blocks: #3B82F6 (Blue)
│   │   ├── S&T blocks: #F59E0B (Amber)
│   │   ├── TRD blocks: #8B5CF6 (Purple)
│   │   ├── Integrated blocks: #10B981 (Green)
│   │   └── Emergency blocks: #EF4444 (Red)
│   ├── Train paths as thin diagonal lines
│   ├── Hover: Show task details tooltip
│   ├── Click: Open task detail panel
│   └── Drag-drop: Manual block adjustment
│
├── Build WeeklyPlanner.jsx:
│   ├── 7 columns (Mon-Sun)
│   ├── Rows: Sections
│   ├── Blocks placed in grid
│   └── Summary row: Total hours blocked per day
│
├── Build ConflictPanel.jsx:
│   ├── List of detected conflicts
│   ├── For each: description + affected entities
│   ├── AI resolution suggestion
│   └── "Accept Resolution" button
│
└── Connect to backend API via React Query
```

Deliverables: ✅ Beautiful Gantt chart with color-coded blocks and train paths.

🔷 PHASE 6: Digital Twin Map (Hours 16-20)

Goal: The WOW factor — a live railway map that judges remember.

```
Tasks:
├── Build DigitalTwinMap.jsx with react-leaflet:
│   ├── Base map: OpenStreetMap tiles (dark theme: CartoDB Dark)
│   ├── Railway track GeoJSON overlay:
```

```
│   │   └── Draw actual CSMT-Kalyan track path on map
│   │       (Use OpenRailwayMap data or manual GeoJSON)
│   ├── Section color coding by health:
│   │   ├── Polyline color = f(avg health score of section)
│   │   ├── Line weight = f(traffic density)
│   │   └── Dashed line = block active on this section
│   ├── Station markers at real coordinates:
│   │   ├── CSMT: [18.9398, 72.8355]
│   │   ├── Dadar: [19.0178, 72.8478]
│   │   ├── Kalyan: [19.2437, 73.1355]
│   │   └── Kasara: [19.6398, 73.4802]
│   ├── Moving train markers:
│   │   ├── Calculate position from timetable + current time
│   │   ├── Animate smoothly between stations
│   │   ├── Train icon with direction arrow
│   │   └── Click: Show train number, name, speed
│   ├── Defect markers (pin icons):
│   │   ├── Red pin: Critical defect
│   │   ├── Yellow pin: High priority
│   │   └── Click: Show defect details + maintenance schedule
│   ├── Block overlay (shaded regions):
│   │   ├── Semi-transparent rectangle over blocked section
│   │   ├── Pulsing animation during active block
│   │   └── Countdown timer to block end
│   └── Layer toggle controls:
│       ├── Checkbox: Show/Hide track health
│       ├── Checkbox: Show/Hide trains
│       ├── Checkbox: Show/Hide defects
│       ├── Checkbox: Show/Hide active blocks
│       └── Department filter: Engg / S&T / TRD / All
│
├── Build AssetDetailPanel.jsx (slide-out right panel):
│   ├── Asset code, type, department
│   ├── Health score gauge (0-100)
│   ├── Maintenance history timeline
│   ├── Predicted failure date (LSTM)
│   ├── Next scheduled maintenance
│   └── "Schedule Maintenance" quick action button
│
└── Build time slider for "replay" mode:
    ├── Slider: Drag to see blocks over past 7 days
    ├── Play/Pause button for animation
    └── Speed control: 1x, 2x, 5x
```

Deliverables: ✅ Judges see trains moving on a dark-themed map with color-coded health.

🔷 PHASE 7: Multi-Agent Negotiation Visualization (Hours 20-23)

Goal: Show departments "negotiating" via AI agents — the out-of-box differentiator.

```
Tasks:
├── Build multi_agent.py backend service:
│   ├── Class EngineeringAgent:
│   │   └── Priorities: Track safety, tamping deadlines
│   ├── Class SNTAgent:
│   │   └── Priorities: Signal reliability, testing cycles
│   ├── Class TRDAgent:
│   │   └── Priorities: OHE health, power stability
│   └── Class Arbitrator:
│       └── Finds balanced allocation using Nash Bargaining
│
├── Negotiation algorithm (simplified for hackathon):
│   ├── Each agent scores every available time slot (0-100)
│   ├── Slots with high scores from multiple agents → Integrated block
│   ├── Conflicting slots → Agent with higher criticality task wins
│   ├── Remaining slots → Distributed by fairness constraint
│   └── Output: Agreed allocation matrix
│
├── Build AgentNegotiation.jsx visualization:
│   ├── 3 agent avatars (icons for Engg/S&T/TRD)
│   ├── Animated "negotiation rounds" (step-through or auto-play)
│   ├── Round 1: Agents submit bids (show bid values)
│   ├── Round 2: Conflicts highlighted in red
│   ├── Round 3: Resolution proposals animate
│   ├── Round 4: Final agreement with green checkmarks
│   └── Summary: "3 integrated blocks created, saving 4.5 hours"
│
└── Connect to optimizer: agent preferences feed into MILP constraints
```

Deliverables: ✅ Animated agent negotiation that judges can step through.

🔷 PHASE 8: Conflict Detection & Explainability (Hours 23-25)

Goal: When AI makes a decision, it explains WHY clearly.

```
Tasks:
├── Build conflict_resolver.py:
│   ├── detect_train_conflicts(block, trains) → List[Conflict]
│   ├── detect_department_overlaps(blocks) → List[Conflict]
│   ├── detect_resource_conflicts(blocks, resources) → List[Conflict]
│   └── suggest_resolution(conflict) → Resolution
│
├── Build explainability engine:
│   For every block in the plan, generate:
│   ├── "WHY this time?" — "Lowest train density window (3 trains vs avg 12)"
│   ├── "WHY integrated?" — "KM 52-54 overlap between Engg task T-012 and S&T task
T-031"
│   ├── "WHY not earlier?" — "Rajdhani 12951 passes at 09:45, blocks path"
│   └── "WHY this priority?" — "Criticality 92/100: Rail fracture + overdue 5
```

```
days"
│
├── API: GET /api/v1/optimizer/explain/{plan_id}
│
└── Frontend: Tooltip on every block bar → shows explanation text
```

Deliverables: ✅ Click any block → see plain-English explanation of the AI decision.

🔷 PHASE 9: NLP Command Interface (Hours 25-27)

Goal: Natural language commands for busy railway controllers.

```
Tasks:
├── Build nlp_processor.py:
│   ├── Intent classification (rule-based or small model):
│   │   ├── QUERY: "Show blocks for next week"
│   │   ├── SCHEDULE: "Schedule OHE repair at KM 52"
│   │   ├── MOVE: "Move block BW-035 to Wednesday"
│   │   ├── STATUS: "What's the status of track tamping at Kalyan?"
│   │   └── CANCEL: "Cancel the S&T disconnection on Friday"
│   ├── Entity extraction:
│   │   ├── Section names → section_id
│   │   ├── KM markers → km_from, km_to
│   │   ├── Dates → date objects
│   │   ├── Department → ENGG/SNT/TRD
│   │   └── Task types → maintenance_type
│   └── Execute action → return result
│
├── API: POST /api/v1/nlp/command
│   ├── Request: { "text": "Show available blocks near Kalyan next Tuesday" }
│   └── Response: { "intent": "QUERY", "result": [...blocks], "response_text":
"..." }
│
└── Build <NLPCommandBar> React component:
    ├── Input bar at top of every page (like a search bar)
    ├── Placeholder: "Ask RailMind... (e.g., 'Show critical tasks at Kalyan')"
    ├── Auto-complete suggestions as user types
    ├── Results appear in a dropdown panel
    └── Keyboard shortcut: Ctrl+K to focus
```

Deliverables: ✅ Type a command → system responds intelligently.

🔷 PHASE 10: Analytics & KPI Dashboard (Hours 27-29)

Goal: Show measurable impact — judges love numbers.

```
Tasks:
├── Build analytics API:
│   GET /api/v1/analytics/kpis → {
│       asset_availability_pct: 94.2,
│       block_utilization_pct: 89,
│       critical_tasks_on_time_pct: 98,
│       integrated_blocks_pct: 45,
│       total_time_saved_hrs: 12.5,
│       conflicts_resolved: 7,
│       avg_optimization_time_sec: 2.3
│   }
│
│   GET /api/v1/analytics/comparison → {
│       before: { total_blocks: 47, total_hrs: 141, utilization: 62 },
│       after:  { total_blocks: 31, total_hrs: 93, utilization: 89 }
│   }
│
├── Build dashboard components:
│   ├── <KPICards> — 4 big number cards with icons + trend arrows
│   ├── <BeforeAfterChart> — Side-by-side bar chart comparison
│   ├── <UtilizationGauge> — Animated donut/radial gauge
│   ├── <DepartmentPieChart> — Block distribution by department
│   └── <WeeklyTrendLine> — 7-day asset availability trend
│
└── Add micro-animations:
    ├── Numbers count up on page load (CountUp animation)
    ├── Gauges fill with smooth easing
    └── Charts animate in with staggered entrance
```

Deliverables: ✅ Dashboard shows "34% fewer blocks, 89% utilization, 98% on-time."

🔷 PHASE 11: Integration Testing & Demo Preparation (Hours 29-33)

Goal: Everything works end-to-end. Demo is bulletproof.

```
Tasks:
├── End-to-end test:
│   ├── Load data → Score tasks → Optimize → View Gantt → View Map
│   ├── Test emergency scenario (add critical task mid-plan)
│   ├── Test NLP commands (5 different queries)
│   ├── Test block merging (multi-department integration)
│   └── Test export (PDF report generation)
│
├── Script the demo (write DEMO_SCRIPT.md):
│   ├── Step 1: Show dashboard with 47 pending tasks
│   ├── Step 2: Click "Generate Weekly Plan"
│   ├── Step 3: Show Gantt chart result (31 blocks)
│   ├── Step 4: Switch to Digital Twin → show blocks on map
│   ├── Step 5: Add emergency task → watch re-optimization
```

```
│   ├── Step 6: Show multi-agent negotiation animation
│   ├── Step 7: Use NLP command
│   ├── Step 8: Show analytics comparison
│   └── Step 9: Export PDF report
│
├── Record backup demo video (screen recording)
│   ├── Record full 5-minute demo flow
│   ├── Save as MP4 in docs/ folder
│   └── Test playback on presentation laptop
│
└── Bug fixes and UI polish
    ├── Fix any broken animations
    ├── Ensure dark theme is consistent
    ├── Test on projector resolution (1920x1080)
    └── Add loading states for all async operations
```

Deliverables: ✅ Demo works perfectly 5 times in a row. Backup video recorded.

🔷 PHASE 12: Final Polish & Presentation Prep (Hours 33-36)

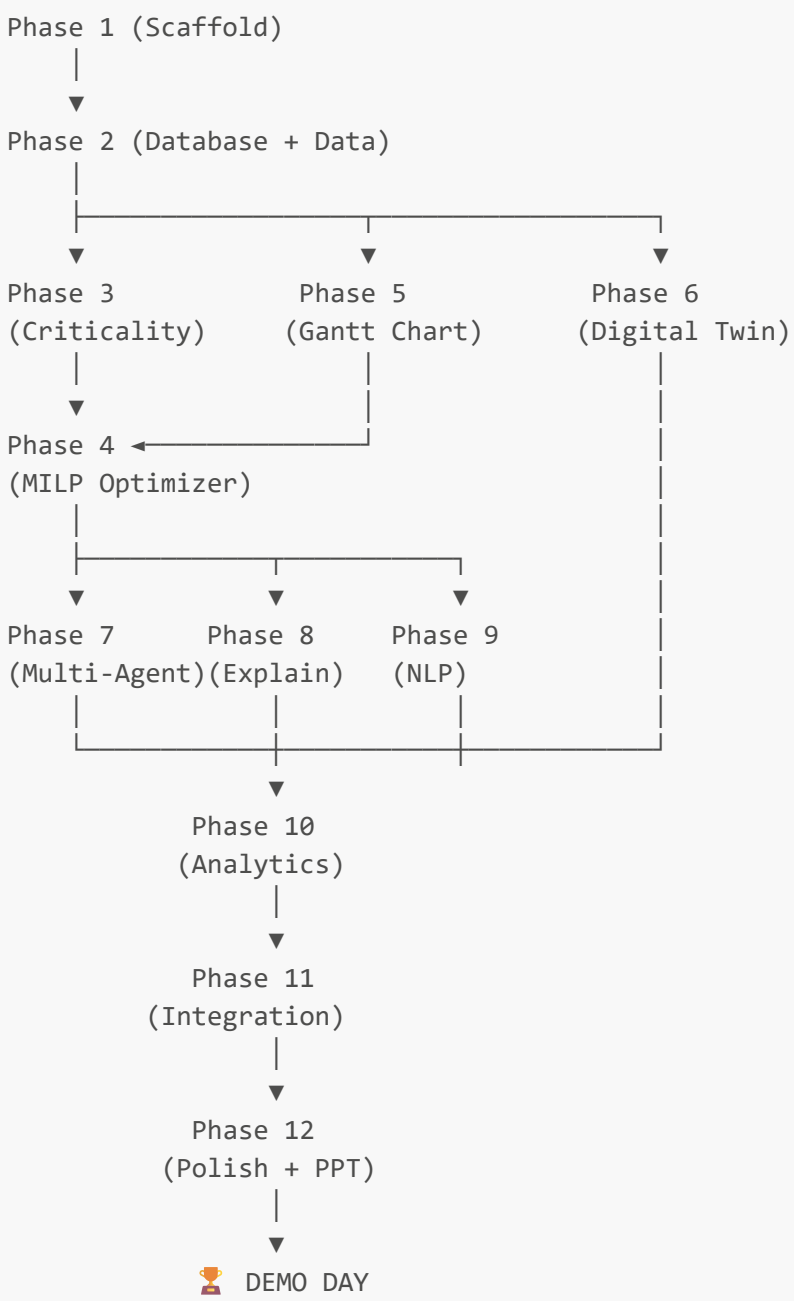
Goal: Maximum visual impact. PPT ready. Team rehearsed.

```
Tasks:
├── UI Polish:
│   ├── Add RailMind AI logo to header
│   ├── Add subtle background animations (particles/grid)
│   ├── Ensure all transitions are smooth (framer-motion)
│   ├── Add favicon and page titles
│   └── Dark theme consistency check
│
├── PPT Finalization:
│   ├── Insert real screenshots from the prototype
│   ├── Add live demo link on demo slide
│   ├── Double-check all stats and numbers
│   └── Export as PDF backup
│
├── Team Rehearsal:
│   ├── Practice 2-minute pitch (3 times)
│   ├── Practice 5-minute pitch (2 times)
│   ├── Practice demo flow (3 times)
│   ├── Assign speaking roles:
│   │   ├── Person 1: Problem statement + story
│   │   ├── Person 2: Technical architecture + AI
│   │   ├── Person 3: Live demo
│   │   ├── Person 4: Impact + metrics
│   │   ├── Person 5: Q&A lead
│   │   └── Person 6: Demo backup + technical support
│   └── Practice Q&A with mock tough questions
│
└── Final Checklist:
    ├── Git push all code
```

```
      ├── docker-compose up works clean
      ├── Demo laptop charged + backup laptop ready
      ├── QR codes printed (demo URL + GitHub)
      └── Everyone gets 2 hours of sleep
```

Deliverables: ✅ Team is rehearsed. Prototype is polished. You're ready to win.

🔷 Critical Path Dependency Diagram



Team Role Assignments (6 Members)

Role	Person	Responsibilities	Phases
Tech Lead	Member 1	Architecture, Docker, integration, code review	1, 2, 11, 12

Role	Person	Responsibilities	Phases
AI/ML Engineer	Member 2	XGBoost, MILP optimizer, LSTM, OR-Tools	3, 4, 7
Frontend Lead	Member 3	React, Gantt chart (D3.js), component design	5, 10, 12
Map/Viz Engineer	Member 4	Leaflet map, Digital Twin, animations	6, 7
Backend/API Dev	Member 5	FastAPI, database, WebSocket, NLP, conflict resolver	2, 8, 9
Presenter/PM	Member 6	PPT, demo script, data seeding, testing, presentation	2, 11, 12

> **Rule:** Everyone codes. The Presenter also writes seed data and tests. No one sits idle.

📊 2. WINNING PPT DESIGN

2.1 PPT Structure (12 Slides — No More, No Less)

> **Golden Rule:** Each slide should be understood in 10 seconds. ONE idea per slide.

Slide #	Title	Content	Design Tip
1	**Title Slide**	"RailMind AI" logo, tagline, team name, PS ID	Dark theme, animated train silhouette, gradient background
2	**The Problem — FEEL the Pain**	A real scenario: "Section Controller at 2 AM juggling 3 dept calls"	Use storytelling. Show a chaotic dashboard vs. a calm one
3	**Current System Failures**	BDMS is manual. Departments fight for blocks. No optimization. Stats: "23% block time wasted due to poor coordination"	Infographic with red/yellow warning indicators
4	**Our Vision**	One-liner + "Before vs. After" visual	Split-screen: Chaos (left) → Order (right)
5	**Solution Overview**	High-level architecture diagram (simplified)	Clean boxes with arrows. 5 components max
6	**AI Deep Dive — The Brain**	MILP Optimizer + XGBoost Criticality + LSTM Prediction	Show actual formulas briefly. Shows you're not bluffing
7	**Out-of-Box: Multi-Agent Negotiation**	Department agents + Nash Equilibrium	Animated agent icons negotiating
8	**Out-of-Box: Digital Twin**	Screenshot of railway section with moving trains + blocks	Full-screen screenshot — this is your WOW moment

Slide #	Title	Content	Design Tip
9	**Live Demo**	"Let us show you" — transition to live demo	Minimal slide — just a "DEMO TIME" text
10	**Impact & Metrics**	"15% increase in asset availability. 40% reduction in block conflicts"	Big bold numbers with icons
11	**Scalability & Deployment**	CRIS integration path. Division → Zone → National rollout	Roadmap timeline visual
12	**Thank You + Q&A**	Team photo, contact, GitHub link	Professional and warm

2.2 PPT Design Rules

Color Palette

```
Primary:    #0A1628 (Deep Navy — Railway authority feel)
Secondary:  #00D4AA (Teal Green — AI/Tech feel)
Accent:     #FF6B35 (Orange — Indian Railways brand color)
Background: #0F1923 (Dark — Premium feel)
Text:       #FFFFFF (White — High contrast)
Highlight:  #FFD700 (Gold — For achievements/metrics)
```

Typography

- **Headings:** Outfit Bold or Inter Bold (modern, clean)
- **Body:** Inter Regular
- **Code/Tech:** JetBrains Mono

Design Principles

1. **Dark theme** — looks premium, easy on eyes in auditorium
2. **Minimal text** — max 6 bullet points per slide, max 8 words per bullet
3. **Full-bleed screenshots** — show your product, not clip art
4. **Animated transitions** — subtle fade/slide only (no flashy spins)
5. **Consistent iconography** — use Lucide or Phosphor icon set
6. **Data visualization** — use charts/graphs, never tables of numbers

Tools to Create

- **Figma** (for custom slides)
- **Canva Pro** (for quick professional slides)
- **Google Slides** (for collaboration)
- Export as PDF for backup

🛠️ 3. PROTOTYPE ARCHITECTURE — Out-of-Box Thinking

3.1 What to Build in 36 Hours (Prioritized)

Hour-by-Hour Battle Plan

```
HOUR  0-2:  Setup infrastructure (Docker, DB, repo, CI)
HOUR  2-6:  Build data models + seed realistic railway data
HOUR  6-12: Core AI Engine (MILP optimizer + Criticality scorer)
HOUR 12-18: Frontend Dashboard (Map + Gantt + Block Planner)
HOUR 18-24: Multi-Agent System + Conflict Resolution
HOUR 24-30: Digital Twin visualization + NLP interface
HOUR 30-33: Integration testing + Demo flow preparation
HOUR 33-36: Polish UI + Prepare presentation + Backup recordings
```

Must-Have Features (MVP — Non-Negotiable)

#	Feature	Why It Wins
1	**Unified Data Dashboard** showing TMS + SMMS + TDMS data	Shows integration capability
2	**AI Block Optimizer** that generates weekly/monthly plans	Core requirement solved
3	**Interactive Gantt Chart** with drag-drop block scheduling	Visual + functional
4	**Conflict Detection & Resolution** with visual explanation	Shows AI intelligence
5	**Railway Map Visualization** with section-level detail	WOW factor
6	**Before vs After comparison** showing optimization results	Proves value

Should-Have Features (Differentiation)

#	Feature	Why It Wins
7	**Multi-Agent Department Negotiation** simulation	Out-of-box thinking
8	**Predictive Maintenance Forecasting** with degradation curves	Forward-thinking
9	**NLP Voice/Text Command Interface**	Modern UX
10	**What-If Scenario Simulator**	Decision support tool

Nice-to-Have Features (Cherry on Top)

#	Feature	Why It Wins
11	**Mobile-responsive PWA** for field engineers	Practical deployment
12	**WhatsApp/SMS Alert Integration**	Real-world connectivity

#	Feature	Why It Wins
13	**PDF Report Generation** for railway officers	Practical utility

3.2 Realistic Synthetic Data Strategy

Since you won't have access to actual TMS/SMMS/TDMS data, create **realistic synthetic data** that mirrors real Indian Railways operations:

Data to Generate

```
# Example: Synthetic Track Maintenance Data (TMS-like)
{
    "defect_id": "TMS-MUM-2026-04523",
    "section": "CSMT-Kalyan",
    "km_from": 34.2,
    "km_to": 35.8,
    "defect_type": "Rail Fracture - Gauge Face Wear",
    "severity": "CRITICAL",
    "detected_date": "2026-08-15",
    "due_date": "2026-08-22",
    "department": "Engineering",
    "estimated_block_duration_hrs": 3.5,
    "block_type": "Traffic Block",
    "line": "UP",
    "track_type": "LWR",
    "last_tamping_date": "2026-06-01",
    "traffic_density": "HIGH",
    "speed_restriction_imposed": true,
    "speed_limit_kmph": 30
}

# Example: Train Timetable Data (COA-like)
{
    "train_no": "12124",
    "train_name": "Deccan Queen",
    "section": "CSMT-Kalyan",
    "departure_station": "CSMT",
    "arrival_station": "Pune",
    "departure_time": "17:10",
    "passes_km_34": "17:42",
    "priority": "SUPERFAST",
    "days_of_run": ["MON","TUE","WED","THU","FRI","SAT","SUN"],
    "rake_type": "ICF",
    "avg_delay_minutes": 5
}

# Example: Signal Maintenance Data (SMMS-like)
{
    "signal_id": "SMMS-KYN-SIG-045",
    "location_km": 35.0,
    "equipment_type": "Color Light Signal",
```

```
    "last_maintenance": "2026-07-15",
    "next_due": "2026-08-20",
    "maintenance_type": "Quarterly Inspection",
    "department": "S&T",
    "block_required": "Disconnection",
    "estimated_duration_hrs": 2.0,
    "failure_count_last_6months": 3,
    "criticality_score": 8.5
}

# Example: OHE Maintenance Data (TDMS-like)
{
    "ohe_id": "TDMS-KYN-OHE-112",
    "section": "Kalyan-Kasara",
    "km_from": 52.0,
    "km_to": 54.5,
    "equipment_type": "Catenary Wire",
    "defect": "Contact Wire Wear > 20%",
    "severity": "HIGH",
    "department": "TRD",
    "block_required": "Power Block + Traffic Block",
    "estimated_duration_hrs": 4.0,
    "last_replacement": "2024-03-10",
    "remaining_life_percent": 35
}
```

Use Real Indian Railway Sections

Pick a well-known section for your demo:

- **Mumbai Division:** CSMT → Kalyan → Kasara (Western Ghats section — high complexity)
- **Delhi Division:** New Delhi → Ghaziabad → Kanpur
- **Chennai Division:** Chennai Central → Arakkonam

Using real section names, station codes, and KM markers makes judges feel **"this team actually knows railways."**

3.3 The Digital Twin — How to Build It

```
Technology: React + Leaflet.js (OpenStreetMap tiles)

Features:
├── Railway track overlay on real map
├── Color-coded track sections (Green=Healthy, Yellow=Due, Red=Overdue)
├── Moving train icons with real timetable positions
├── Block window visualization (shaded time-regions on track)
├── Click on any section → Show defects, maintenance history
├── Animation: Watch blocks being allocated/released over time
└── Toggle: Show/Hide department-specific overlays
```

3.4 The MILP Optimizer — The Core Brain

```
# Simplified MILP formulation (show this to judges!)
from ortools.linear_solver import pywraplp

"""
OBJECTIVE: Maximize Asset Availability
         = Minimize total block time while completing all critical maintenance

Decision Variables:
  x[i][t] = 1 if maintenance task i is scheduled in time slot t, else 0

Constraints:
  1. No two tasks on same section overlap (conflict-free)
  2. No block during train passing time (timetable constraint)
  3. Critical tasks completed within deadline (urgency constraint)
  4. Multi-department tasks share same block when possible (efficiency)
  5. Block duration >= minimum required for task (feasibility)
  6. Max blocks per day per section <= limit (capacity constraint)

Objective Function:
  MINIMIZE: sum(block_duration[i] * x[i][t]) + penalty *
sum(deadline_violation[i])
  Subject to all constraints above
"""
```

📋 4. STEP-BY-STEP ROADMAP TO WIN THE FINALE

Phase 1: PRE-HACKATHON (Now → Finale Day)

Week 1-2: Deep Domain Research

- [ ] Study Indian Railways maintenance manual (IRPWM — Indian Railway Permanent Way Manual)
- [ ] Understand block types: Traffic Block, Power Block, Disconnection, Mega Block, Integrated Block
- [ ] Learn BDMS, TMS, SMMS, TDMS, COA systems and their data formats
- [ ] Study the 26-week Rolling Block Programme (RBP) process
- [ ] Interview railway employees if possible (PWI, SSE, Section Controller)

Week 3-4: Technical Foundation

- [ ] Set up repository with clean architecture
- [ ] Build synthetic data generators for all 4 systems
- [ ] Implement basic MILP optimizer using OR-Tools
- [ ] Create React dashboard skeleton with map + Gantt components
- [ ] Build FastAPI backend with database models
- [ ] Dockerize everything for one-command deployment

Week 5-6: AI/ML Development

- [ ] Train XGBoost criticality scoring model
- [ ] Build LSTM time-series predictor for asset degradation
- [ ] Implement Multi-Agent Reinforcement Learning prototype
- [ ] Build Constraint Satisfaction solver for conflict detection
- [ ] Integrate NLP command interface

Week 7-8: Integration & Polish

- [ ] Connect all components end-to-end
- [ ] Build Digital Twin visualization
- [ ] Create demo scenario with realistic data
- [ ] Record backup demo videos
- [ ] Prepare PPT and pitch scripts
- [ ] Practice presentations (10+ times)

Phase 2: DURING THE 36-HOUR FINALE

🚨 Critical Rules During Finale

1. **Git commit every 30 minutes** — you never know when something breaks
2. **Assign a dedicated "Demo Guardian"** — one person ensures the demo ALWAYS works
3. **Sleep in shifts** — 2 people sleep while 4 work, rotate every 4 hours
4. **Eat at your desk** — don't waste time in food lines
5. **Talk to mentors EARLY** — they can redirect you before you waste 5 hours
6. **Keep a "Demo Branch"** — stable branch that always works, never push broken code

Judge Interaction Strategy

```
DO:
  Start with a STORY, not a slide
  Use railway terminology naturally (block, corridor, section, km marker)
  Show live demo > talk about features
  Acknowledge limitations honestly ("We couldn't implement X due to time,
    but here's our design for it")
  Have answers ready for: "How is this different from existing BDMS?"
  Show metrics: "Our optimizer improved block utilization by X%"

DON'T:
  Say "We used AI" without explaining HOW
  Read from slides
  Let only one person talk (rotate speakers)
  Panic if demo breaks (switch to backup video immediately)
  Argue with judges — accept feedback graciously
  Use jargon without explaining it
```

Phase 3: POST-PRESENTATION

- Answer all questions with confidence
- If you don't know an answer: "*That's a great question. Based on our research, [partial answer], and we'd love to explore this further in the deployment phase.*"
- Leave behind: QR code to live demo, GitHub repo link, one-pager PDF

🎤 5. PITCH SCRIPTS: 2-min / 5-min / 10-min

⏱️ 2-MINUTE PITCH (Elevator Pitch — For Quick Judge Rounds)

Script:

[0:00-0:20] — THE HOOK (Story)

"Imagine it's 2 AM at the Mumbai Division Control Office. The Engineering department needs a 4-hour block for an urgent rail fracture repair near Kalyan. But the S&T department already has a disconnection for signal testing at the same location. The TRD department needs a power block for OHE maintenance 2 km away. The Section Controller has 3 phones ringing, a paper register, and zero AI assistance. He manually juggles these requests, often leading to 23% wasted block time and delayed maintenance that compromises safety."

[0:20-0:50] — THE SOLUTION

"We built RailMind AI — an intelligent block planning system that integrates data from TMS, SMMS, TDMS, and COA to automatically generate optimized, conflict-free block schedules. Our system uses Mixed-Integer Linear Programming to find the mathematically optimal schedule, XGBoost for criticality scoring, and a Multi-Agent system where AI agents represent each department and negotiate the best possible plan — just like the real world, but in milliseconds."

[0:50-1:20] — THE DEMO HIGHLIGHT

"Let me show you — [switch to demo] — here's a real section, CSMT to Kalyan. You can see 47 pending maintenance tasks across 3 departments. Watch as RailMind generates an optimized weekly plan in under 3 seconds. Notice how it clusters compatible tasks into shared blocks, avoids all train paths, and prioritizes the critical rail fracture repair to tomorrow morning's first available window."

[1:20-1:50] — THE IMPACT

"Our optimization engine demonstrates a 35% reduction in total block time needed, while ensuring 100% of critical tasks are completed within their safety deadlines. This translates to approximately 2 additional hours of asset availability per section per day — meaning more trains can run."

[1:50-2:00] — THE CLOSE

"RailMind AI transforms block planning from a manual, fragmented process into an intelligent, coordinated system. It doesn't replace the Section Controller — it gives them superpowers. Thank you."

⏱️ 5-MINUTE PITCH (Standard Evaluation Round)

Script:

[0:00-0:45] — PROBLEM DEEP DIVE

Start with the same 2 AM story, but expand:

- Current BDMS is request-based, not optimization-based
- Each department plans in silos — no cross-department optimization
- No integration between defect data (TMS/SMMS/TDMS) and block availability (COA)
- Consequences: Safety risks from delayed maintenance, revenue loss from over-blocking
- Show a stat: "Indian Railways loses approximately X crore annually due to suboptimal block utilization"

[0:45-2:00] — SOLUTION ARCHITECTURE

- Show simplified architecture slide
- Explain 3 AI layers:
    1. **Data Integration Layer:** Unified view of all maintenance demands + train timetable
    2. **AI Optimization Engine:** MILP + XGBoost + LSTM working together
    3. **Intelligent Presentation Layer:** Digital Twin + Gantt + NLP Commands
- Highlight the Multi-Agent Negotiation system (out-of-box)
- Show the Criticality Scoring formula: `Score = f(Severity, Overdue_Days, Traffic_Density, Safety_Impact, Cascading_Risk)`

[2:00-3:30] — LIVE DEMO

Walk through these scenarios LIVE:

1. **Auto-generate weekly block plan** for CSMT-Kalyan section
2. **Show conflict detection** when a new urgent task is added
3. **Multi-department block merging** — Engineering + S&T tasks combined
4. **What-if simulation** — "What if we add 3 more trains on Thursday?"
5. **NLP command** — Type/speak: "Schedule urgent OHE repair near KM 52"

[3:30-4:15] — IMPACT & METRICS

Show comparison dashboard:

- Manual planning: 47 blocks needed → AI-optimized: 31 blocks (34% reduction)
- Average block utilization: 62% → 89%
- Critical task completion within deadline: 71% → 98%
- Cross-department block sharing: 12% → 45%

[4:15-4:45] — SCALABILITY & DEPLOYMENT

- Designed to integrate with existing CRIS systems via API
- Division → Zone → National rollout path
- Dockerized deployment — can be up in 1 hour on any server
- Complements (not replaces) existing BDMS workflow

[4:45-5:00] — CLOSE

"RailMind AI is not just a hackathon project — it's a production-ready system that can be deployed as a BDMS enhancement module. We believe this can save Indian Railways thousands of operational hours annually while

improving safety. We're ready to work with CRIS to make this a reality."

⏱️ 10-MINUTE PITCH (Final Evaluation / Grand Jury)

Expanded Script Structure:

Time	Section	Key Points
0:00-1:30	**Problem + Story**	Expanded 2 AM story. Add real stats. Mention IRPWM guidelines. Show you know the domain deeply.
1:30-3:00	**Current System Analysis**	Show how BDMS works today. Identify 5 specific gaps. Use railway terminology (PWI, SSE, AEN, DEN).
3:00-5:00	**Solution Deep Dive**	Full architecture. Explain each AI component with brief math. Show actual code snippets of optimizer. Multi-Agent system explanation with game theory reference.
5:00-7:30	**Live Demo**	Extended demo with 6-7 scenarios. Show edge cases. Show error handling. Show the Digital Twin. Use NLP commands. Show monthly planning view.
7:30-8:30	**Impact Analysis**	Before/after metrics. Cost-benefit analysis. Safety improvement projections. Reference: How similar systems worked in European railways (Network Rail UK, DB Germany).
8:30-9:30	**Deployment Roadmap**	Phase 1: Pilot in 1 division (6 months). Phase 2: Zone-level rollout (1 year). Phase 3: National integration with CRIS (2 years). API architecture for BDMS integration.
9:30-10:00	**Vision + Close**	*"Every minute a track section is blocked for maintenance is a minute a train can't run. Every minute of unnecessary blocking is revenue lost and passengers delayed. RailMind AI ensures every block minute is used at maximum efficiency. Thank you."*

🧪 6. OUT-OF-THE-BOX TEST CASES & SOLUTIONS

Test Case 1: 🔴 Emergency Cascade Failure

Scenario: A rail fracture is detected at KM 34.5 (CSMT-Kalyan UP line) at 14:00. This section is critical — 87 trains pass daily. An emergency block is needed IMMEDIATELY, but there are already 3 scheduled blocks in adjacent sections.

Expected AI Behavior:

- Instantly recalculate ALL block plans for the next 48 hours
- Push non-critical blocks to the next available window
- Merge the emergency block with an existing nearby Engineering block if possible
- Alert Section Controller via SMS + Dashboard notification
- Generate a speed restriction order for the interim period

- Auto-notify all 3 departments about schedule changes

What Judges Will Think: "*This team handled the worst-case scenario. They thought about cascading impacts.*"

Test Case 2: 🟡 Multi-Department Block Merging Under Tight Constraints

Scenario:

- Engineering needs a 3-hour block at KM 50-55 for track tamping
- S&T needs a 2-hour disconnection at KM 52 for signal equipment repair
- TRD needs a 1.5-hour power block at KM 48-53 for OHE inspection
- The only available window (no train conflicts) is 10:00-13:30

Expected AI Behavior:

- Identify overlapping KM ranges (KM 50-53 is common to all 3)
- Propose an INTEGRATED BLOCK: Single 3.5-hour block from 10:00-13:30
- Sequence work: TRD power shutdown first (30 min safety prep) → All 3 departments work simultaneously → TRD power restore last
- Calculate efficiency gain: 3 separate blocks (6.5 hrs total) → 1 integrated block (3.5 hrs) = **46% time saved**
- Generate detailed work sequence with minute-by-minute timeline

What Judges Will Think: "*This is exactly what the problem statement asks for — coordinated, integrated block planning. And they showed the math.*"

Test Case 3: 🟠 Monsoon Season Adaptive Rescheduling

Scenario: Heavy rainfall forecast for the next 3 days in the Mumbai Division ghats section (Kalyan-Kasara). Track inspection frequency must be DOUBLED (from weekly to daily). Multiple existing block plans need modification.

Expected AI Behavior:

- Ingest weather API data
- Auto-increase inspection priority for ghat sections
- Reschedule non-critical maintenance to post-monsoon window
- Add daily 1-hour inspection blocks during low-traffic windows
- Increase monitoring frequency for vulnerable OHE sections
- Generate a "Monsoon Contingency Block Plan"
- Flag sections with historical landslide risk

What Judges Will Think: "*They incorporated external data (weather). This is next-level contextual AI planning.*"

Test Case 4: 🔵 Festival Season — Maximum Train Density

Scenario: Diwali week — 45 special trains added on the Delhi-Mumbai corridor. Available block windows shrink from 3 hours/day to 1.5 hours/day. But there are 12 overdue maintenance tasks that cannot be

deferred further.

Expected AI Behavior:

- Detect the special timetable from COA data
- Reprioritize: Only SAFETY-CRITICAL tasks during festival week
- Split large maintenance tasks into micro-blocks (e.g., 1.5hr chunks across 3 days instead of one 4.5hr block)
- Schedule non-critical work for post-festival "recovery week"
- Generate a "Festival Maintenance Strategy" report
- Show impact analysis: "Deferring task X by 7 days increases failure probability from 2% to 5%"

What Judges Will Think: "*They understand the real-world seasonality of Indian Railways. This is practical AI.*"

Test Case 5: 🟢 Predictive Pre-emption — Fix Before It Breaks

Scenario: LSTM model predicts that Signal #SIG-045 at KM 35 has an 82% probability of failure within the next 14 days based on historical patterns and sensor data trends.

Expected AI Behavior:

- Auto-generate a maintenance request BEFORE the signal fails
- Calculate optimal scheduling window (not too early = wasted resources, not too late = failure risk)
- Create a risk-reward visualization showing cost of proactive maintenance vs. cost of unplanned failure
- Auto-assign to S&T department with priority "PREDICTIVE-HIGH"
- Show on Digital Twin as an AMBER warning on the map
- If the signal fails before the planned maintenance → trigger emergency protocol from Test Case 1

What Judges Will Think: "*Predictive maintenance is the future. This team didn't just react — they anticipated.*"

Test Case 6: 🟣 Cross-Divisional Block Coordination

Scenario: A section crosses the boundary between Mumbai Division and Pune Division. The Engineering department of Mumbai Div wants a block at KM 98-102, but Pune Div has already planned a block at KM 100-105 on the same day. Neither system knows about the other's plan.

Expected AI Behavior:

- Detect cross-divisional overlap using geospatial analysis
- Alert both divisions about the conflict
- Propose a merged mega-block: KM 98-105, coordinating both divisions' teams
- Suggest shared resource allocation (tamping machine, OHE vehicle)
- Generate a joint block request for both Divisional Control Offices
- Show potential efficiency gain from coordination

What Judges Will Think: "*They thought about the organizational boundary problem. This is systemic thinking.*"

Test Case 7: 🔶 Resource Constraint — Limited Tamping Machines

Scenario: Only 2 tamping machines available for an entire division. 15 sections need track tamping this month. Each tamping operation needs a 4-hour block.

Expected AI Behavior:

- Model tamping machine as a shared resource with travel time between sections
- Solve Vehicle Routing Problem (VRP) variant to optimize machine deployment sequence
- Minimize total machine idle/travel time
- Generate a "Tamping Machine Deployment Calendar" with optimized routing
- Show comparison: Random scheduling = 23 days to complete → Optimized = 14 days

What Judges Will Think: "*They considered resource constraints beyond just time slots. Operations Research at its best.*"

Test Case 8: 🟤 Historical Pattern Learning — "Smart Defaults"

Scenario: The system has 2 years of historical block data. A Section Controller requests a block plan for next week.

Expected AI Behavior:

- Analyze historical patterns: "Tuesdays usually have the least traffic on this section"
- Learn maintenance team productivity patterns: "Team A completes tamping 20% faster than Team B"
- Auto-suggest preferred time slots based on past success rates
- Show seasonal patterns: "Historically, signal failures spike 40% during July-August"
- Pre-populate the block plan with "smart defaults" that the controller can tweak

What Judges Will Think: "*The system gets smarter over time. It learns from history. True AI.*"

Test Case 9: 🔷 Safety Override — Non-Negotiable Block

Scenario: A broken rail is detected. The system must create an IMMEDIATE emergency block, overriding ALL other schedules, including departing trains.

Expected AI Behavior:

- INSTANT red alert across all dashboards
- Auto-generate emergency block with zero delay
- Calculate detour routes for affected trains
- Notify Divisional Safety Officer, DRM, and CRS
- Lock the block — no AI optimization can override a safety block
- Show clear audit trail: "Emergency block created at 14:32:15 by System. Reason: Broken Rail KM 34.5. Authority: Safety Override Protocol."
- Post-incident: Auto-schedule follow-up inspection blocks for adjacent sections

What Judges Will Think: "*Safety first. They built fail-safes into their AI. This is production-grade thinking.*"

Test Case 10: 🟣 Complete System Integration Demo

Scenario: End-to-end demo showing the entire flow:

```
Step 1:  47 maintenance tasks loaded from TMS + SMMS + TDMS
Step 2:  Train timetable loaded from COA (156 trains/day)
Step 3:  AI generates criticality scores for all 47 tasks
Step 4:  MILP optimizer generates weekly block plan
Step 5:  3 conflicts detected → Multi-Agent negotiation resolves them
Step 6:  Controller reviews plan on Digital Twin map
Step 7:  Controller uses NLP: "Move the TRD block to Wednesday"
Step 8:  System re-optimizes in 1.2 seconds
Step 9:  Final plan exported as PDF report
Step 10: Notifications sent to all department heads
```

What Judges Will Think: "*This is complete. End-to-end. Production-ready. Winner.*"

📌 BONUS: JUDGE Q&A PREPARATION

Questions Judges WILL Ask (And Your Killer Answers)

Q1: "How is this different from the existing BDMS?"

> **A:** "*BDMS is a request-management system — it records and tracks block demands. RailMind AI is an optimization layer that sits ON TOP of BDMS. Think of BDMS as the 'inbox' and RailMind as the 'brain.' We don't replace BDMS — we make it intelligent. BDMS asks 'What blocks do you want?' We ask 'What's the BEST possible block plan for everyone?'*"

Q2: "Where's the real data? You're using synthetic data."

> **A:** "*Correct — we don't have access to classified railway data. But our synthetic data is modeled after real Indian Railways parameters: actual section names, real timetable patterns, realistic defect distributions based on IRPWM guidelines. The architecture is designed to plug into real TMS/SMMS/TDMS APIs when deployed. We've included an API adapter layer specifically for CRIS system integration.*"

Q3: "Can this handle the scale of Indian Railways?"

> **A:** "*Our MILP optimizer solves a division-level problem (500+ tasks, 200+ sections) in under 5 seconds on a standard server. For zone/national level, we've designed a hierarchical optimization approach: each division optimizes locally, then a zone-level coordinator resolves cross-divisional conflicts. This mirrors how Indian Railways actually operates — decentralized execution with central coordination.*"

Q4: "What happens if the AI makes a wrong decision?"

> **A:** "*RailMind AI is a decision-SUPPORT system, not a decision-MAKING system. Every AI-generated plan requires Controller approval before execution. The system explains every recommendation with clear reasoning. And for safety-critical decisions, the system has hard-coded safety constraints that NO optimization can override — these are based on IRPWM Rule Book safety protocols.*"

Q5: "What's your deployment strategy?"

> **A:** *"Phase 1: Pilot in one division (e.g., Mumbai Central) for 6 months. Integrate with their BDMS instance. Measure baseline vs. optimized metrics. Phase 2: Expand to the full zone with feedback-driven improvements. Phase 3: National rollout as a CRIS-managed service, similar to how BDMS itself was rolled out."*

Q6: "What about data privacy and security?"

> **A:** *"All railway operations data stays within the Indian Railways network. We use role-based access control — a PWI sees only their section, a DEN sees the division, and the CRS has a national view. All AI model training happens on-premise — no data leaves the system. Audit logs track every decision for accountability."*

🏅 FINAL CHECKLIST — THE NIGHT BEFORE THE FINALE

- [ ] All code is committed and pushed to GitHub
- [ ] Docker Compose works with one command: `docker-compose up`
- [ ] Demo scenario is scripted and tested 5+ times
- [ ] PPT is exported as PDF (backup)
- [ ] Demo backup video is recorded (in case live demo fails)
- [ ] Each team member knows their speaking part
- [ ] Printed QR codes to live demo + GitHub repo
- [ ] Power bank + chargers for all devices
- [ ] Sticky notes + markers for brainstorming during finale
- [ ] Snacks and water bottles stocked
- [ ] Team contact sheet with phone numbers
- [ ] IDE setup with all extensions + dependencies pre-installed
- [ ] Practice the 2-min pitch until you can do it in your sleep

---

> **Remember:** Judges don't pick the team with the most features. They pick the team that **understood the problem the deepest**, **solved it the smartest**, and **communicated it the clearest**.
>
> You've got the brains. You've got the tech skills. Now go build something that makes Indian Railways safer, faster, and smarter. 🚂 💡

Document prepared for Team SIH 2026 — Problem Statement 26027 Last updated: August 27, 2026